

Retail25

Software Stack Documentation

A self-hosted, web-based Point of Sale, Inventory and Accounts-Receivable platform, rebuilt from *Retail Plus for Windows 2.5* (True North Computer Services, 2008), with bulk RFID reading added.

Document	Software Stack Documentation
Product version	0.1.0 (backend/Directory.Build.props)
Repository branch	main
Commit	e7f91ad
Generated	9 October 2026
Purpose	Technology inventory, architecture and deployment reference

Stack at a glance. C# on .NET 10 with Clean Architecture and CQRS (MediatR); SQL Server via Entity Framework Core 10; ASP.NET Core Web API with SignalR for realtime; ASP.NET Core Identity with OpenIddict and JWT bearer schemes; Next.js 14 (App Router) with React 18, TypeScript and Tailwind CSS as a backend-for-frontend; and a Windows-service terminal agent that drives RFID readers and till peripherals.

Every technology, version and component in this document was read from a file in the repository. Items that could not be established from the repository are labelled *Not Verified* rather than inferred.

Contents

1	Executive Summary	1
1.1	What the system is made of	1
1.2	How data is stored and moved	1
1.3	How it is built and shipped	1
1.4	Important caveat for the reader	2
2	Technology Selection — What, Why, and Purpose	3
3	Overall System Architecture	5
3.1	Notes on the diagram	5
4	Application Runtime Flow	6
4.1	The RFID path is different	8
5	Frontend Technology	9
5.1	Framework and rendering	9
5.2	Screen inventory	9
5.3	Layers	9
5.4	Routing, forms and accessibility	10
6	Backend Technology	11
6.1	Runtime and host	11
6.2	Project layout and dependency rule	11
6.3	Request handling	11
6.4	Cross-cutting services	11
7	Programming Languages	13
8	Frameworks	14
9	Libraries and Important Dependencies	15
9.1	Backend, by purpose	15
9.2	Security pins	16
10	Database and Data Layer	17
10.1	Engine and access	17
10.2	Domain areas	17
10.3	The ledger decision	17
10.4	Secrets	18

11 Authentication and Authorization	19
11.1 What is actually implemented	19
11.2 Session handling in the browser	19
11.3 Authorization	19
11.4 The hub ticket	22
12 API Technology	23
12.1 Style and shape	23
12.2 Endpoint groups	23
12.3 Realtime hubs	23
12.4 Conventions	23
13 Build Tools and Package Management	25
14 Testing	26
14.1 What exists	26
14.2 Configured versus executed	26
14.3 A gap worth recording	26
15 Deployment and CI/CD	27
15.1 Continuous integration	27
15.2 Release pipeline	27
15.3 Container and host options	27
15.4 The terminal agent is not part of either pipeline	28
16 Hosting Requirements	29
17 Important Dependencies and Risk Assessment	31
17.1 Required for the application to start	31
17.2 Single points of failure	31
17.3 Security posture observed	31
17.4 Risks requiring investigation, not asserted as faults	32
18 Master Technology Stack Table	33
18.1 Front-end dependency inventory	34
19 Technology Interaction Matrix	36
20 Repository Evidence and Verification Notes	38
20.1 Where this document was read from	38
20.2 Principal files inspected	38
20.3 Documentation divergences found	39
20.4 Not found, not configured, or unable to verify	39
20.5 Inference, flagged	39
21 Summary and Next Steps	40
21.1 The confirmed stack	40
21.2 Architectural characteristics worth noting	40
21.3 What is missing or unverified	40
21.4 Recommended next steps	40

List of Figures

3.1	System architecture	5
4.1	Application runtime flow	7
10.1	RFID topology model	18
11.1	Authentication and authorization flow	21
12.1	API request lifecycle	24
15.1	Release pipeline	27

List of Tables

2.1	Principal technologies, their role, and the evidence for each	3
9.1	Backend dependencies by purpose	15
16.1	Hosting and runtime requirements	29
18.1	Master technology stack	33
18.2	Front-end dependencies: declared range versus locked version	34
19.1	Component interactions	36

Chapter 1

Executive Summary

Retail25 replaces a 2008 Windows desktop application — *Retail Plus for Windows 2.5* — with a self-hosted web system covering point of sale, inventory, and accounts receivable. The legacy system stored data in DBF files on mapped network drives; Retail25 replaces that with SQL Server, an ASP.NET Core API and a Next.js front end, and adds bulk RFID reading that the original did not have.

1.1 What the system is made of

Five runnable pieces, each verified in the repository:

- **A Web API** (`backend/src/Retail25.Api`) — ASP.NET Core controllers under `api/v1/...`, four SignalR hubs, health endpoints and background jobs.
- **A domain and application core** (`Retail25.Domain`, `Retail25.Application`) — entities, the pricing engine, and CQRS handlers dispatched by Mediatr.
- **An infrastructure layer** (`Retail25.Infrastructure`) — EF Core against SQL Server, ASP.NET Core Identity, OpenIddict, caching and realtime notifiers.
- **A web front end** (`frontend`) — Next.js 14 App Router. It is not only a UI: its route handlers act as a *backend for frontend*, holding the session cookie and proxying to the API so no token ever reaches the browser.
- **A terminal agent** (`Retail25.TerminalAgent`) — a Windows service installed on a till that drives RFID readers, scales, cash drawers and pole displays, and connects *outbound* to the server over SignalR.

1.2 How data is stored and moved

Persistence is SQL Server through EF Core 10 with snake-case naming; the `ApplicationDbContext` exposes 89 entity sets across 12 migrations. Cart state, tag debouncing and idempotency records are held through a cache provider that defaults to SQL Server, with Redis as an explicit opt-in. Browser traffic is HTTPS to the Next.js server, which calls the API server-to-server; live updates reach the browser over SignalR WebSockets, authenticated by a single-use ticket rather than a token.

1.3 How it is built and shipped

Continuous integration (`.github/workflows/ci.yml`) runs four jobs — backend tests, frontend checks, end-to-end browser tests and container image builds. A separate pipeline (`.github/`

`workflows/deploy-myasp.yml`) tests, publishes the API, builds the front end, applies database migrations and deploys to shared Windows hosting. Container definitions and a Compose file also exist for self-hosting.

1.4 Important caveat for the reader

Parts of the project's own documentation are out of date with respect to the code, and this document follows the code. The most consequential divergences are recorded in Chapter 20: the database is **SQL Server, not PostgreSQL**; staff sign-in is a **custom JWT endpoint, not the Authorization Code + PKCE flow** described in the architecture dossier; and the parity benchmark that `CLAUDE.md` describes as authoritative **does not exist on this branch**.

Chapter 2

Technology Selection — What, Why, and Purpose

The repository rarely records *why* a technology was chosen. Where a rationale is written down — usually in a code comment or in `README.md` — it is quoted and attributed. Where it is not, this chapter describes the technology’s observable role and the practical benefit it brings in that role, and does not claim to know the original intent.

Table 2.1: Principal technologies, their role, and the evidence for each

Technology	What it is	Responsibility here	Why it suits that role	Evidence
.NET 10 / C#	Application runtime and language	All server code, the agent, and the migration tool	One runtime for the server and for a Windows service that must open serial ports	<code>Directory.Build.props: net10.0, LangVersion 14.0</code>
ASP.NET Core	Web framework	HTTP API, SignalR hubs, health checks, rate limiting	Hosts the API and the realtime hubs in one process	<code>Retail25.Api/Program.cs</code>
MediatR 12.4.1	In-process mediator	Dispatches commands and queries to handlers	Keeps controllers thin and the application layer free of transport concerns	<code>Directory.Packages.props</code>
EF Core 10.0.10	ORM	Schema, migrations, queries	Explicit migrations give a reviewable schema history	<code>Directory.Packages.props</code> ; 12 files in <code>Persistence/Migrations</code>
SQL Server	Relational database	All persistent state	Chosen in place of PostgreSQL; <code>README</code> records the swap	<code>UseSqlServer</code> in <code>Infrastructure/DependencyInjection.cs:83</code>
ASP.NET Core Identity	Membership system	Users, roles, password hashing, lockout	Supplies the credential checks the custom token endpoint calls	<code>CheckPassword</code> <code>SignInAsync</code> in <code>StaffAuthController.cs:96</code>

continued on next page

Table 2.1 continued from previous page

Technology	What it is	Responsibility here	Why it suits that role	Evidence
OpenIddict 7.6.0	OAuth2 / OIDC server	Token endpoint for machine clients and authorization-code flow	Lets the terminal agent authenticate as a client, separate from staff	Identity Registration.cs:154-163
SignalR	Realtime transport	Four hubs for POS, inventory, terminals and RFID	Pushes tag reads and grid changes without polling	Program.cs:423--429
Next.js 14.2.35	React framework	UI <i>and</i> backend-for-frontend route handlers	Server-side routes hold the session so no token reaches the browser	frontend/package.json; app/api/auth/sign-in/route.ts
React 18.3.1	UI library	Component rendering	—	package-lock.json
TypeScript 5.9.3	Typed JavaScript	All front-end source	—	package-lock.json; tsconfig.json
Tailwind CSS 3.4.19	Utility CSS framework	Styling	—	tailwind.config.js
Radix UI	Headless component primitives	Dialogs, menus, tabs, toasts, tooltips	Accessible behaviour without imposed visuals	ten @radix-ui/* entries in package.json
TanStack Query 5.101.4	Server-state cache	Fetching and caching API data	—	package-lock.json
Zustand 5.0.14	Client state store	POS and UI state	—	frontend/src/stores/
QuestPDF 2024.10.3	PDF generation	Receipts and documents	Produces documents server-side without a print driver	Directory.Packages.props
Hangfire 1.8.24	Background jobs	Late-charge accrual, accounting posting	Durable scheduling backed by the same SQL Server	Program.cs:439--445
Serilog 10.0.0	Structured logging	Application and request logs	—	UseSerilog Program.cs:31
OpenTelemetry 1.17.0	Telemetry	Traces and metrics via OTLP	—	Program.cs:38; deploy/otel/config.yaml
Argon2 1.3.1	Password hashing	Staff PIN hashing	Memory-hard hashing for short PINs	Argon2PinHasher

Chapter 3

Overall System Architecture

Figure 3.1 shows the components found in the repository and how they communicate. Solid arrows are interactions established by source code. Dashed arrows mark optional or configuration-dependent paths.

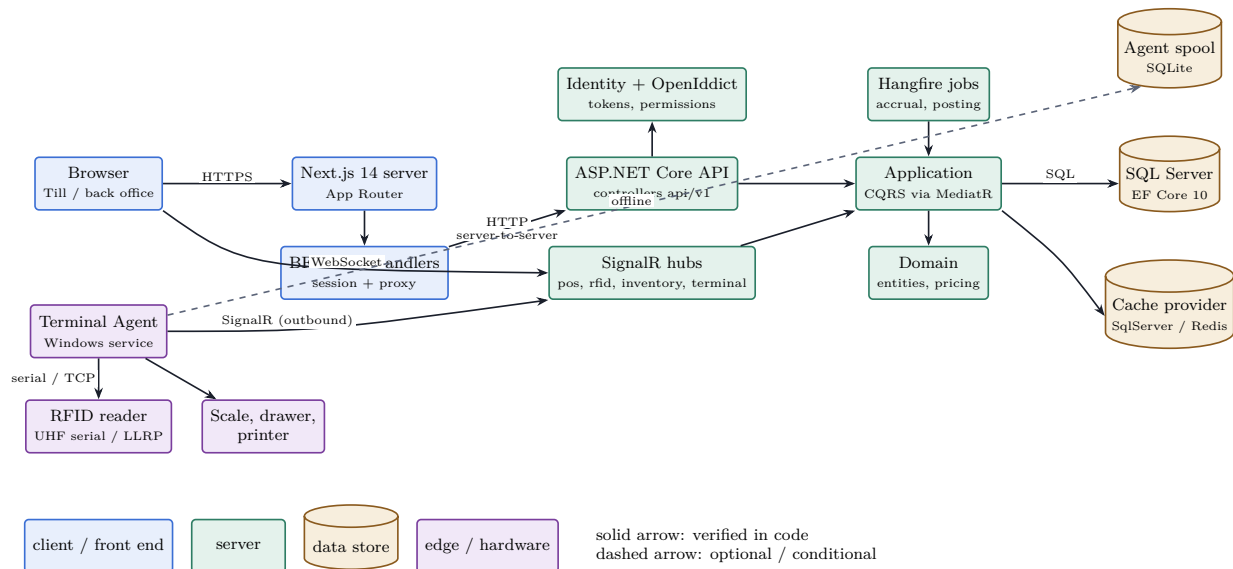


Figure 3.1: System architecture as found in the repository. The browser never calls the API directly: it talks to the Next.js server, whose route handlers hold the session and proxy onward. The terminal agent connects *outbound*, so a till needs no inbound firewall rule.

3.1 Notes on the diagram

- **The browser holds no token.** The session is an encrypted, `httpOnly` cookie written by the Next.js server (`frontend/src/lib/server/session.ts`). API calls go through `app/api/proxy/[...path]/route.ts`, which attaches the token server-side.
- **SignalR is the exception to that rule**, and is handled deliberately: the browser is issued a *single-use hub ticket* from `app/api/auth/hub-ticket/route.ts` rather than a bearer token, because a WebSocket handshake cannot carry an `Authorization` header.
- **The agent dials out.** It is a SignalR *client* of `/hubs/terminal`.
- **The cache is pluggable.** `Cache:Provider` selects SQL Server, Redis or in-memory; the dashed path reflects that Redis is opt-in.

Chapter 4

Application Runtime Flow

Figure 4.1 traces a request from the browser through the system. It reflects the implemented path: the browser's own fetch goes to the Next.js server, not to the API.

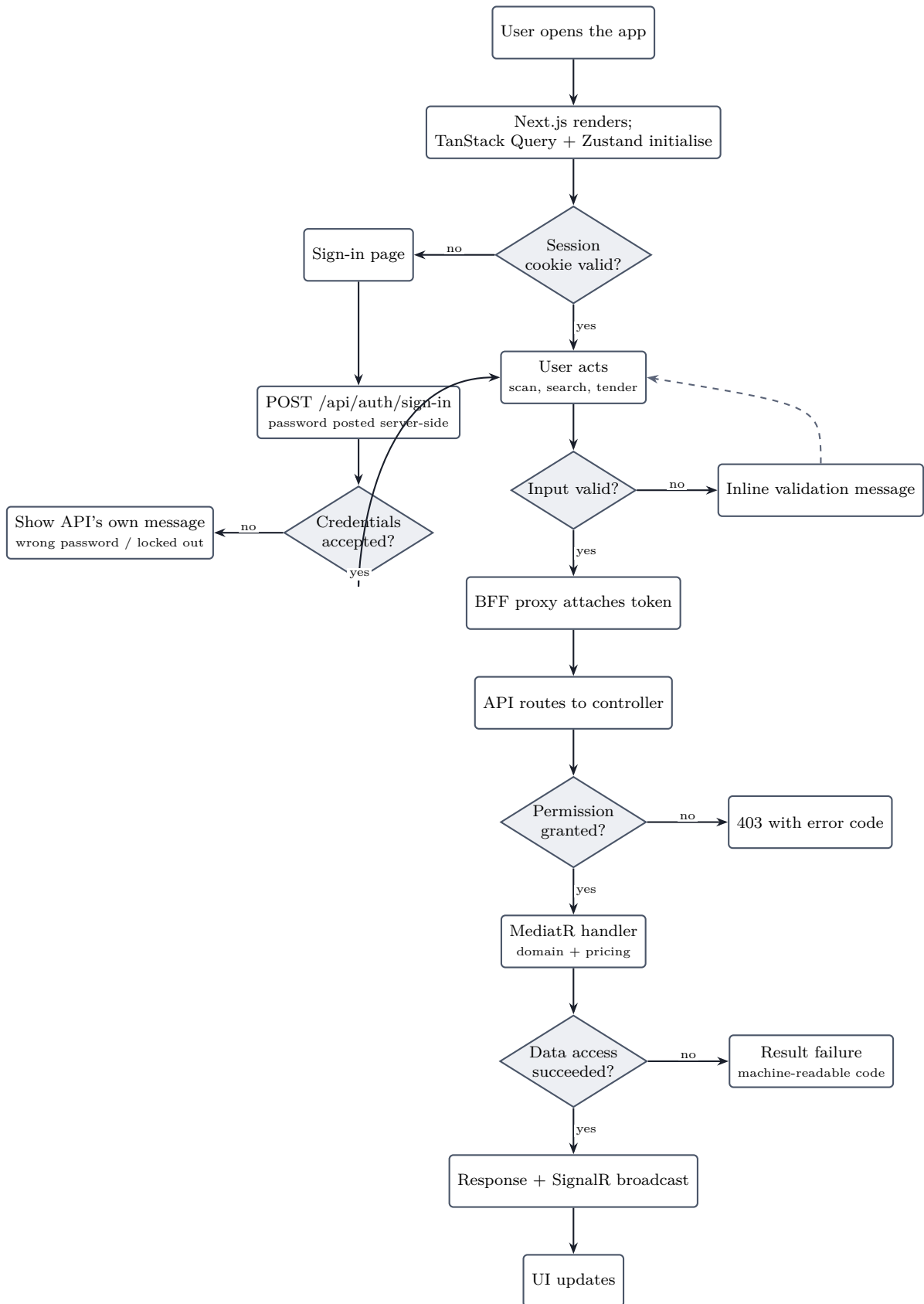


Figure 4.1: Runtime flow for a browser-initiated action. Not every request needs authorization or the database — read-only lookups short-circuit after the handler — so the decision nodes are branches, not mandatory stages.

4.1 The RFID path is different

A tag read does not begin in the browser. The agent reads tags from the reader, debounces them locally, and publishes them to `/hubs/terminal`. The server resolves which till the read belongs to from the reader and antenna, prices it onto that till's cart, and notifies the till over `/hubs/rfid`. The browser is a spectator in this path.

Chapter 5

Frontend Technology

5.1 Framework and rendering

Next.js **14.2.35** (pinned exactly, not a range) using the **App Router** (`frontend/src/app/`). `next.config.js` sets two options, both load-bearing:

- `reactStrictMode: true` — React’s development double-invoke is active.
- `output: 'standalone'` — emits `.next/standalone/server.js` with only the modules actually reached. The file’s own comment gives the reason: shared hosting, where uploading a full `node_modules` tree "is measured in hours".

5.2 Screen inventory

Routes under `src/app/(dashboard)/`: `admin`, `catalog`, `customers`, `dashboard`, `help`, `inventory`, `orders`, `pos`, `purchasing`, `receivables`, `reports`, `sales`. Outside the dashboard group: `sign-in`, `sign-up`, `forgot-password`, `reset-password`, plus `error.tsx`, `global-error.tsx` and `not-found.tsx` error boundaries.

5.3 Layers

Concern	Technology	Evidence
Server state	TanStack Query 5.101.4	<code>package-lock.json</code> ; devtools also present
Client state	Zustand 5.0.14	<code>src/stores/pos-store.ts</code> , <code>ui-store.ts</code>
Styling	Tailwind CSS 3.4.19 + PostCSS 8.5.23 + Autoprefixer 10.5.4	<code>tailwind.config.js</code> , <code>postcss.config.js</code>
Class composition	<code>clsx 2.1.1</code> , <code>tailwind-merge 2.6.1</code> , <code>class-variance-authority 0.7.1</code>	<code>package.json</code>
Components	Radix UI primitives (dialog, dropdown-menu, label, select, separator, slot, tabs, toast, tooltip)	ten <code>@radix-ui/*</code> dependencies
Icons	<code>lucide-react 0.468.0</code>	<code>package.json</code>
Charts	<code>recharts 2.15.4</code>	<code>package.json</code>

Long lists	@tanstack/react-	used by the POS product grid
	virtual 3.14.9	
HTTP client	axios 1.18.1	src/lib/api-client.ts
Realtime	@microsoft/signalr	src/lib/rfid-hub.ts, inventory-hub.ts
	8.0.17	
Dates	date-fns 4.4.0	package.json
Crypto (session seal)	jose 5.10.0	src/lib/server/session.ts

5.4 Routing, forms and accessibility

Routing is file-system based (App Router). **No dedicated form library** (React Hook Form, Formik) and **no schema validation library** (Zod, Yup) is declared in `package.json`; forms are handled with React state and server-side validation — recorded here as *Not Found* rather than assumed. Accessibility rests on Radix primitives, which supply roles and keyboard behaviour; no separate accessibility tooling (axe, pally) is configured.

Chapter 6

Backend Technology

6.1 Runtime and host

.NET 10 (`net10.0`), C# `LangVersion 14.0`, with `Nullable` enabled and `TreatWarningsAsErrors` set — so a warning fails the build. Analysers are on at `latest-recommended`.

Entry point: `backend/src/Retail25.Api/Program.cs`. It configures, in order: Serilog hosting, OpenTelemetry, health checks, controllers, Swagger, SignalR, CORS, authentication, rate limiting, response compression, then the middleware pipeline and endpoint mapping.

6.2 Project layout and dependency rule

Project	Responsibility
<code>Retail25.Domain</code>	Entities, value objects, the pricing engine. No project references.
<code>Retail25.Application</code>	Commands, queries, handlers, ports. Depends on Domain and Contracts.
<code>Retail25.Infrastructure</code>	EF Core, Identity, OpenIddict, caching, realtime notifiers. Implements Application ports.
<code>Retail25.Api</code>	Controllers, SignalR hubs, host configuration.
<code>Retail25.Contracts</code>	Shared DTOs between server and agent.
<code>Retail25.Devices</code>	RFID and peripheral drivers (UHF serial, LLRP, simulator).
<code>Retail25.TerminalAgent</code>	Windows service running on a till.
<code>Retail25.Migration</code>	Legacy DBF and CSV importer.

The direction of dependencies is enforced by a test, not only by convention: `tests/Retail25.ArchitectureTests/DependencyRuleTests.cs` uses `NetArchTest`.

6.3 Request handling

Controllers are thin and dispatch through MediatR. Expected failures are returned as a `Result` carrying a stable machine-readable code rather than thrown — visible throughout the application layer and converted to HTTP status codes by a shared `ToActionResult` extension.

6.4 Cross-cutting services

- **Rate limiting** — three named policies in `Program.cs:305--307`: `auth` (20), `pin` (10) and `lookup` (300) per caller. `MapControllers().RequireRateLimiting("lookup")`.
- **Health checks** — `/health/live` and `/health/ready` (`Program.cs:415--416`), with SQL Server and Redis checks registered.

-
- **Background jobs** — Hangfire, with two recurring jobs resolved from DI: `LateChargeAccrualJob` and `PostPosRevenueToAccountingJob` (`Program.cs:439--445`).
 - **Response compression** and **Serilog request logging** are both enabled.

Chapter 7

Programming Languages

Language	Where	Evidence
C#	All .NET projects	*.csproj; LangVersion 14.0 in Directory.Build.props
TypeScript	All front-end source	tsconfig.json; typescript 5.9.3 locked
JavaScript	Build configuration	next.config.js, tailwind.config.js, postcss.config.js
SQL	Schema reference and diagnostics	docs/architecture/schema.sql, docs/runbooks/diagnostic-queries.sql
CSS	Global styles and Tailwind layers	src/app/globals.css
YAML	CI/CD and collector config	.github/workflows/*.yaml, deploy/otel/config.yaml, deploy/docker-compose.yaml
PowerShell	Operational scripts	update-till-agent.ps1 and installer scripts
XML	Project and host configuration	*.csproj, deploy/iis/frontend.web.config
Dockerfile	Image definitions	deploy/Dockerfile.api, deploy/Dockerfile.web

Chapter 8

Frameworks

Framework	Version	Role and configuration
ASP.NET Core	10.0.x	API, hubs, health, rate limiting. Part of the shared framework; version tracks net10.0 .
Entity Framework Core	10.0.10	ORM. SQL Server provider, snake-case naming via <code>EFCore.NamingConventions 10.0.1</code> , explicit migrations.
MediatR	12.4.1	CQRS dispatch.
FluentValidation	11.11.0	Request validation, with DI extensions.
OpenIddict	7.6.0	OAuth2/OIDC server. Pinned to 7.x because its EF Core integration binds to the EF Core major (comment in <code>Directory.Packages.props</code>).
ASP.NET Core Identity	10.0.10	Users, roles, lockout, password hashing.
Hangfire	1.8.24	Background jobs on SQL Server storage.
Next.js	14.2.35	React framework, App Router, standalone output.
React	18.3.1	UI rendering.
Tailwind CSS	3.4.19	Utility CSS.
xUnit	2.9.3	Test framework for all five test projects.
Playwright	1.62.0	End-to-end browser tests.
Vitest	2.1.9	Front-end unit tests (<code>environment: 'node'</code>).

Chapter 9

Libraries and Important Dependencies

All .NET versions below come from `backend/Directory.Packages.props`, which uses Central Package Management — every version is pinned exactly, so declared and effective versions are the same. Front-end versions are the *locked* versions from `frontend/package-lock.json` (lockfile version 3).

9.1 Backend, by purpose

Table 9.1: Backend dependencies by purpose

Package	Version	Why it is here
<i>Application core</i>		
MediatR	12.4.1	Command/query dispatch
FluentValidation (+ DI ext.)	11.11.0	Request validation
Mapster	7.4.0	Object mapping
<i>Persistence</i>		
Microsoft. EntityFrameworkCore	10.0.10	ORM core
.Relational / .SqlServer / .Design	10.0.10	Provider and migration tooling
.InMemory	10.0.10	Test provider
EFCore.NamingConventions	10.0.1	snake_case table and column names
Microsoft.Data.Sqlite	10.0.10	The agent's offline tag spool
<i>Identity and security</i>		
Identity. EntityFrameworkCore	10.0.10	User and role stores
DataProtection. EntityFrameworkCore	10.0.10	Keys persisted to the database, so they survive a recycle
Authentication.JwtBearer	10.0.10	Staff and shopper bearer schemes
OpenIddict.AspNetCore	7.6.0	Authorization and token endpoints
OpenIddict. EntityFrameworkCore	7.6.0	Client and token storage
OpenIddict.Validation. AspNetCore	7.6.0	Token validation
Konscious...Argon2	1.3.1	PIN hashing
<i>Caching and realtime</i>		
StackExchange.Redis	2.8.16	Redis client (opt-in)

continued on next page

Package	Version	Why it is here
SignalR. StackExchangeRedis	10.0.10	Hub backplane when Redis is used
SignalR.Client	10.0.10	Used by the terminal agent
<i>Jobs, documents, data</i>		
Hangfire.AspNetCore / .SqlServer	1.8.24	Recurring jobs
QuestPDF	2024.10.3	Receipts and PDF documents
ZXing.Net	0.16.9	Barcode encoding/decoding
ClosedXML	0.104.2	Excel export
CsvHelper	33.0.1	CSV import/export
YamlDotNet	16.2.1	YAML parsing
<i>Observability</i>		
Serilog.AspNetCore	10.0.0	Structured logging
Serilog sinks: Console / File	6.1.1 / 7.0.0	Log destinations
OpenTelemetry. Extensions.Hosting	1.17.0	Telemetry host integration
OTLP exporter; ASP.NET Core, Http, Runtime instrumentation	1.17.0	Traces and metrics
<i>API surface</i>		
Swashbuckle.AspNetCore	10.2.3	Swagger/OpenAPI
Microsoft.AspNetCore. OpenApi	10.0.10	OpenAPI metadata
HealthChecks.SqlServer / .Redis	9.0.0	Readiness probes
<i>Hardware</i>		
System.IO.Ports	10.0.10	Serial access for readers and peripherals
Hosting.WindowsServices	10.0.10	Runs the agent as a Windows service

9.2 Security pins

Directory.Packages.props carries a block of transitive pins that exist solely to clear advisories, each annotated in the file with its GHSA identifier:

Package	Pinned	Advisory / arrives via
Newtonsoft.Json	13.0.3	GHSA-5crp-9r3c-p9vr; under Hangfire
SQLitePCLRaw (lib, bundle, core, provider)	2.1.12	GHSA-2m69-gcr7-jv3q; under Microsoft.Data.Sqlite
MessagePack	2.5.302	GHSA-4qm4-8hg2-g2xm; SignalR hub protocol

The file records the reasoning: each is pinned to the lowest patched version *inside its existing major*, because taking the newest would mean major bumps "across a serial-port spool and a hub protocol, for no security benefit over these."

Chapter 10

Database and Data Layer

10.1 Engine and access

Aspect	Finding
Engine	Microsoft SQL Server. UseSqlServer at Infrastructure/DependencyInjection.cs:83
Container image	mcr.microsoft.com/mssql/server:2022-latest (deploy/docker-compose.yml)
ORM	EF Core 10.0.10
Naming	snake_case via UseSnakeCaseNamingConvention()
Context	ApplicationDbContext, 89 DbSet properties
Configurations	28 files in Persistence/Configurations/
Migrations	12, latest 20260819081314_DeviceEnrolment
Applied	On start when Database:AutoMigrate is set
Design-time	DesignTimeDbContextFactory for dotnet ef
Secondary store	SQLite (Microsoft.Data.Sqlite) — the agent's offline tag spool

10.2 Domain areas

Retail25.Domain is organised by aggregate area: Accounting, Catalog, Common, Configuration, Customers, Identification, Inventory, Migration, Orders, Purchasing, Receivables, Sales, Security, Shoppers, Staff, Terminals, Trolleys, ValueObjects.

10.3 The ledger decision

README.md states the governing rule: *"The transaction is a ledger, not a row."* Sales, stock movements, AR balances and drawer totals are append-only, with derived snapshots — `Product.OnHand` is a projection of `stock_ledger_entries` rather than an independently mutated number. The README attributes this to the legacy system's DBF mutation and its need for Rebuild/Reindex recovery.

A full entity-relationship diagram is not reproduced here: with 89 entity sets it would be unreadable at page size, and a generated reference already exists at `docs/architecture/schema.sql` and `docs/architecture/12-schema-reference.md`. Figure 10.1 shows the one sub-model that this document's RFID discussion depends on.

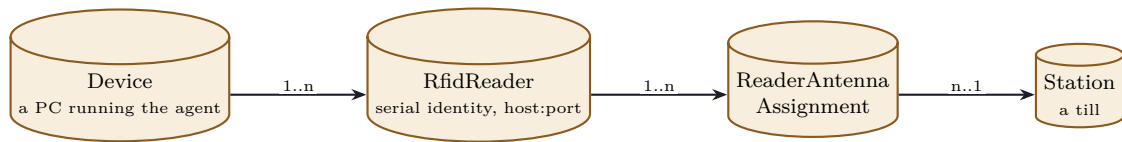


Figure 10.1: The RFID topology sub-model. One machine drives many readers; each reader’s antennas are mapped individually to tills, so one four-antenna reader can serve four separate checkouts. Introduced by migration 20260819071121_RfidTopology.

10.4 Secrets

No connection string, password or key is reproduced in this document. Connection strings are supplied through configuration (`ConnectionStrings:DefaultConnection`); the deployment workflow injects the staff token signing key as a secret at publish time.

Chapter 11

Authentication and Authorization

11.1 What is actually implemented

This is the area where the project's documentation and its code diverge most, so the findings are stated plainly with evidence.

Audience	Mechanism
Staff (till, back office)	Custom JWT endpoint. POST /auth/token in StaffAuthController.cs:74. Credentials are checked with ASP.NET Core Identity's CheckPasswordSignInAsync(..., lockoutOnFailure: true) (line 96) and a token pair is issued by StaffTokenIssuer. Also POST /auth/refresh and GET /auth/me.
Shopper (phone app)	Separate JwtBearer scheme, ShopperAuthentication.Scheme (IdentityRegistration.cs:319), with its own signing key and no permission claims.
Machine clients (terminal agent)	OpenIddict client_credentials against connect/token.

OpenIddict is configured with AllowAuthorizationCodeFlow(), AllowRefreshTokenFlow() and AllowClientCredentialsFlow() (IdentityRegistration.cs:161--163), exposing connect/authorize and connect/token. The authorization-code flow therefore *exists*; it is simply not what staff sign-in uses any more. frontend/src/app/api/auth/sign-in/route.ts records the move in its own comment: the password "is posted to the API from the server", which keeps the "no token is reachable from JavaScript" guarantee "true after moving off the redirect flow".

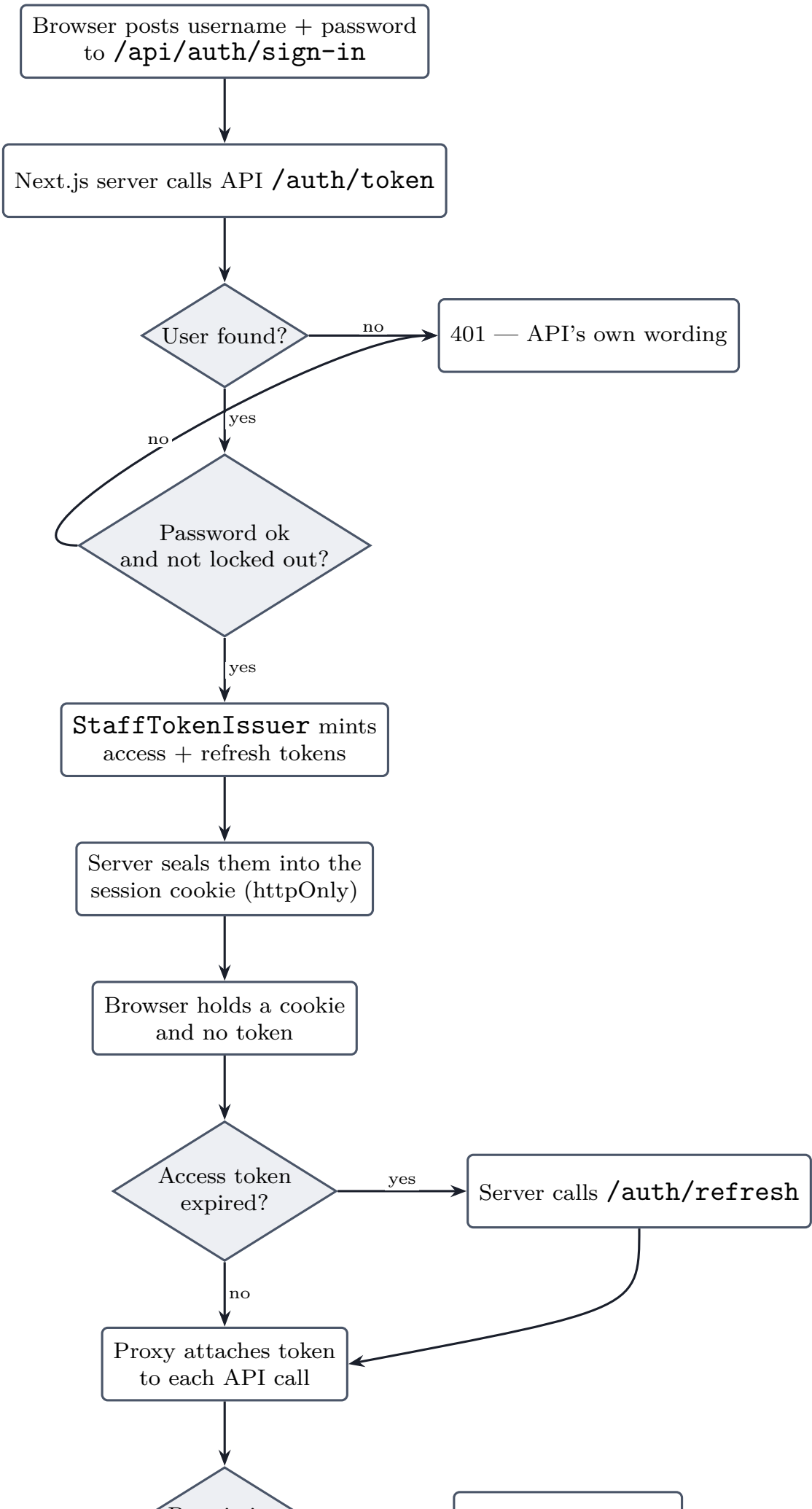
11.2 Session handling in the browser

- One cookie, written by the Next.js server: httpOnly, SameSite=Lax, and Secure in production.
- Named __Host-r25.session in production and r25.session otherwise. The code explains the split: the __Host- prefix requires Secure, which plain-HTTP development cannot satisfy, and a browser silently discards a prefixed cookie that fails the contract.
- Contents are sealed with jose 5.10.0 before being written.
- Data Protection keys are persisted to the database (DataProtection.EntityFrameworkCore), so an application-pool recycle does not invalidate every session.

11.3 Authorization

Permissions are data, not code: permission and role_permission rows resolved by a PermissionResolver, with a PermissionClaim carried on the identity. Hub access uses a ded-

icated policy (`HubAuthorizationPolicy`) that requires an authenticated user. PINs are hashed with Argon2 (`Argon2PinHasher`).



11.4 The hub ticket

A WebSocket handshake cannot carry an `Authorization` header, and handing the browser a bearer token would defeat the backend-for-frontend. The system issues a *single-use hub ticket* instead (`app/api/auth/hub-ticket/route.ts` → `api/v1/hub-tickets`), redeemed by `HubTicketMiddleware` and exchanged for a claims identity. The route's own comment describes the ticket as "deliberately almost worthless: it authenticates one hub connection, expires in a minute, and is consumed on use."

Chapter 12

API Technology

12.1 Style and shape

A REST-style HTTP API of ASP.NET Core controllers, versioned by path segment (`api/v1/...`), alongside four SignalR hubs for realtime. OpenAPI is produced by Swashbuckle 10.2.3 (`SwaggerDoc("v1", ...Title = "Retail25 API"), Program.cs:98`).

12.2 Endpoint groups

Forty route prefixes were found. Grouped by area:

Area	Route prefixes under <code>api/v1/</code>
Selling	<code>carts</code> , <code>sales</code> , <code>price-quotes</code> , <code>tender-types</code> , <code>drawer-sessions</code> , <code>layaways</code> , <code>customer-orders</code> , <code>gift-cards</code>
Catalogue & stock	<code>catalog</code> , <code>catalog/bulk</code> , <code>products</code> , <code>products/{id}/matrix</code> , <code>inventory</code> , <code>stock-counts</code> , <code>transfers</code> , <code>serialized-units</code>
Purchasing & finance	<code>purchase-orders</code> , <code>suppliers</code> , <code>receivables</code> , <code>fiscal-years</code> , <code>sync/accounting</code>
People	<code>customers</code> , <code>staff</code> , <code>loyalty</code> , <code>account</code> , <code>approvals</code>
Hardware	<code>terminals</code> , <code>rfid-topology</code> , <code>hub-tickets</code>
Shopper app	<code>shopper</code> , <code>shopper/account</code>
Platform	<code>settings</code> , <code>reports</code> , <code>documents</code> , <code>audit</code> , <code>diagnostics</code> , <code>maintenance</code> , <code>migration</code> , <code>locations/{id}/branding</code>
Outside <code>api/v1</code>	<code>auth</code> (staff tokens), <code>account</code> (interactive pages), <code>connect/*</code> (OpenIdDict)

12.3 Realtime hubs

Hub	Purpose
<code>/hubs/pos</code>	Till state and sale updates
<code>/hubs/inventory</code>	Back-office grid patches
<code>/hubs/terminal</code>	The agent's own channel (tags in, commands out)
<code>/hubs/rfid</code>	Tag observations and reader status to the till

12.4 Conventions

Errors return a problem document with a stable code (for example `device.not_found`, `validation.failed`) rather than an English sentence, so the UI can translate. Rate limiting is applied per caller with the three policies named in Chapter 6. CORS exposes a single named policy, `Frontend`, whose origins come from `AllowedOrigins`.

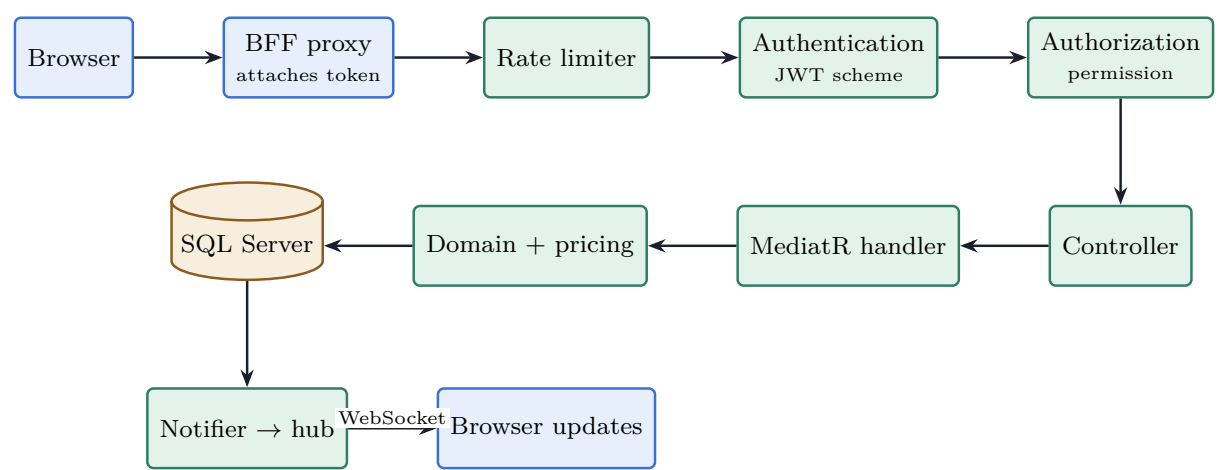


Figure 12.1: Lifecycle of an API request, from the browser through the backend-for-frontend proxy to the database, and the realtime notification that follows a state change.

Chapter 13

Build Tools and Package Management

Concern	Tool	Evidence
.NET package manager	NuGet, <i>Central Package Management</i>	Directory.Packages.props, ManagePackageVersionsCentrally, CentralPackageTransitivePinningEnabled
.NET build	dotnet build / dotnet publish	Retail25.sln; CI and deploy workflows
.NET compiler settings	net10.0, C# 14, warnings as errors	Directory.Build.props
Node package manager	npm	package-lock.json, lockfile version 3; CI uses npm ci
Front-end build	next build (standalone output)	package.json scripts; next.config.js
Dev server	next dev	package.json
Type checking	tsc --noEmit	typecheck script
Linting	ESLint 8.57.1 + eslint-config-next 14.2.21	lint script
CSS pipeline	PostCSS 8.5.23, Autoprefixer 10.5.4, Tailwind 3.4.19	postcss.config.js

No .NET-side formatter configuration (.editorconfig aside) and no Prettier configuration were found; code formatting for the front end rests on ESLint. Recorded as *Not Found*.

Chapter 14

Testing

14.1 What exists

Project / suite	Files	Kind and tooling
Retail25.Domain. UnitTests	15	Pricing and tax. Property-based testing with CsCheck 4.2.0.
Retail25.Application. UnitTests	52	Handler and service behaviour; NSubstitute 5.3.0 for doubles.
Retail25.TerminalAgent. UnitTests	16	Agent, codecs, spool.
Retail25. ArchitectureTests	2	Dependency rules (NetArchTest 1.3.2) and shipped transport-security settings.
Retail25. IntegrationTests	24	Real SQL Server and Redis via Testcontainers 4.0.0; Mvc.Testing for in-process hosting.
frontend unit	—	Vitest 2.1.9, environment: 'node', src/**/*.test.ts(x).
frontend/e2e	5 specs	Playwright 1.62.0 against a running stack.

Coverage collection is available through `coverlet.collector 6.0.2`. Static analysis is the .NET analyser set at `latest-recommended` with warnings as errors, plus ESLint on the front end.

14.2 Configured versus executed

Everything above is *configured*. This document makes no claim about current pass rates: no test run was performed as part of generating it, and CI results are not reproduced here. A reviewer wanting that evidence should consult the most recent run of `.github/workflows/ci.yml`.

14.3 A gap worth recording

CLAUDE.md (present on another branch, see Chapter 20) describes a *parity benchmark* of 155 legacy behaviours, generated into `docs/BENCHMARK.md` and driven by `Benchmark/capabilities.json`. **Neither file exists on this branch**, and `Retail25.ArchitectureTests` contains only the two suites listed above. The parity-tracking mechanism described in that document is therefore *Not Found* here.

Chapter 15

Deployment and CI/CD

15.1 Continuous integration

`.github/workflows/ci.yml` defines four jobs: `backend`, `frontend`, `e2e` and `containers`. The `backend` job restores, builds with warnings as errors, runs the unit, agent and architecture suites, then the integration suite, and asserts that the integration suite actually executed rather than skipping. The `e2e` job starts SQL Server and Redis as services, generates throwaway token-signing keys, starts the API, installs Playwright and runs the browser suite. The `containers` job builds both images and pushes them to GHCR only from `main`.

15.2 Release pipeline

`.github/workflows/deploy-myasp.yml` targets shared Windows hosting and defines five jobs: `test`, `backend`, `frontend`, `database` and `deploy`. Notable steps visible in the file: restoring a hand-maintained `web.config`, injecting the staff token signing key as a secret, and proving the sub-application configuration and the assembled front-end tree are intact before publishing.

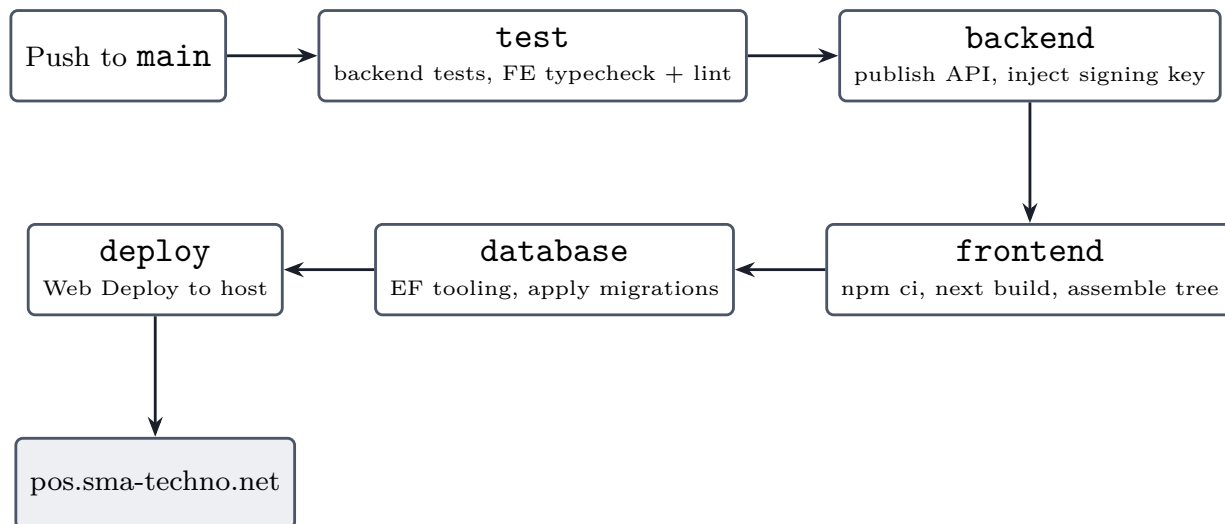


Figure 15.1: The release pipeline to shared Windows hosting. Job names are taken verbatim from `.github/workflows/deploy-myasp.yml`; the arrows show the order in which the file declares them and should not be read as a guarantee of strict sequencing.

15.3 Container and host options

Artefact	Content
deploy/Dockerfile. api	Build on mcr.microsoft.com/dotnet/sdk:10.0; run on aspnet:10.0; EXPOSE 8080; entrypoint dotnet Retail25.Api.dll
deploy/Dockerfile. web	Three stages on node:20-alpine; EXPOSE 3000; CMD ["node", "server. js"] (standalone output)
deploy/docker- compose.yml	Services sqlserver (mssql/server:2022-latest), redis (redis:7- alpine), otel-collector (otel/opentelemetry-collector-contrib:0. 115.1), api, web; volumes sqldata, redisdata
deploy/caddy/ Caddyfile	Reverse proxy configuration
deploy/iis/ frontend.web.config	IIS host configuration for the front end
deploy/otel/config. yaml	Collector pipeline

15.4 The terminal agent is not part of either pipeline

The agent is installed on a till by hand and is not produced or deployed by CI. It is a Windows service (`Hosting.WindowsServices`) with `StartType=Automatic` and failure-restart actions, and it connects outbound, so a till needs no inbound firewall rule.

Chapter 16

Hosting Requirements

Each requirement carries a label: **Verified** established by a project file; **Derived** reasonably inferred from the implementation; **Recommended** a proposed practice, not a project requirement; **Not Verified** insufficient evidence.

Table 16.1: Hosting and runtime requirements

Requirement	Status	Detail and evidence
.NET runtime	Verified	ASP.NET Core 10 runtime; <code>net10.0</code> in <code>Directory.Build.props</code>
Node runtime	Verified	Node 20 for the front end; <code>node:20-alpine</code> in <code>Dockerfile.web</code>
Database	Verified	SQL Server; image <code>mssql/server:2022-latest</code> in Compose
Redis	Verified	Optional. Only when <code>Cache:Provider</code> is Redis; <code>redis:7-alpine</code>
API port	Verified	8080 in the container (EXPOSE 8080)
Web port	Verified	3000 in the container (EXPOSE 3000)
HTTPS	Verified	A startup guard refuses to run with the shipped placeholder public origin; <code>__Host-cookies</code> require <code>Secure</code>
Public origin configuration	Verified	<code>Auth:WebOrigin</code> and <code>AllowedOrigins</code> must be set, or the API refuses to start
Token signing keys	Verified	Outside Development, OpenIddict refuses to start without signing and encryption certificates
Database migration on start	Verified	Applied when <code>Database:AutoMigrate</code> is set
Windows for the terminal agent	Verified	<code>Hosting.WindowsServices</code> ; serial ports via <code>System.IO.Ports</code>
Network path to readers	Derived	The agent opens TCP to readers, or a serial port; readers must be reachable from the till
Persistent storage	Derived	Database volume (<code>sqldata</code>) and the agent's SQLite spool under <code>ProgramData</code>
Outbound HTTPS from tills	Derived	The agent dials the server; no inbound rule needed
Telemetry endpoint	Derived	OTLP exporter configured; a collector is provided in Compose
CPU / RAM / disk sizing	Not Verified	No deployment manifest, autoscaling policy or load test in the repository states a figure. Capacity must be established by load testing; this document will not invent one.

continued on next page

Requirement	Status	Detail and evidence
Backup and recovery	Recommended	A restore runbook exists (<code>docs/runbooks/restore.md</code>); no automated backup schedule is defined in the repository
Log aggregation	Recommended	Serilog writes console and file sinks; shipping them elsewhere is not configured

Chapter 17

Important Dependencies and Risk Assessment

17.1 Required for the application to start

- **SQL Server.** Everything persists here, and Hangfire uses it for job storage. Unavailable means the API does not serve.
- **A valid public-origin configuration.** The API deliberately refuses to start when `Auth:WebOrigin` or `AllowedOrigins` still carry the shipped `...example` placeholder.
- **Token signing and encryption keys** outside Development, for the same reason — a deliberate refusal rather than a silent fallback.
- **A session secret** for the front end: the BFF "refuses to hold a session without it" (`frontend/.env.example`).

17.2 Single points of failure

Point	Consequence and mitigation found
SQL Server	Total outage. No replica or failover configuration is present in the repository.
Cache provider SqlServer	The file's own comment states there is no SignalR backplane under SQL Server, so this mode means a single API instance . Running more than one requires Redis.
Terminal agent per till	If it stops, that till reads no tags. It has automatic restart actions, and an offline SQLite spool so reads survive a server outage.
One client per RFID reader	A reader accepts a single TCP client, so the agent and any vendor tool are mutually exclusive. This is a hardware property, not a defect.

17.3 Security posture observed

Positives found in the code: warnings-as-errors with analysers on; a transitive security-pin block with advisory identifiers; a vulnerable-package scan step in CI (`dotnet list package --vulnerable --include-transitive`); Argon2 for PINs; cookies that are `httpOnly`, `SameSite` and `__Host-` prefixed in production; single-use, short-lived hub tickets instead of browser-held bearer tokens; and a startup guard against an unconfigured public origin.

17.4 Risks requiring investigation, not asserted as faults

- **Documentation drift.** Three significant divergences are catalogued in Chapter 20. Drift of this kind is itself an operational risk: an engineer following `docs/architecture/` would configure PostgreSQL and expect a PKCE flow.
- **No parity benchmark on this branch.** The mechanism that was supposed to prevent a feature being "marked delivered by editing a document" is absent here.
- **npm audit runs with continue-on-error** in CI, so front-end advisories do not fail the build. Whether any are currently outstanding was not assessed.
- **Node 20** is the front-end runtime; its support horizon should be checked against the project's own timeline. No end-of-life claim is made here.

Chapter 18

Master Technology Stack Table

Versions below are exact where the repository pins them. .NET packages come from `Directory.Packages.props` (central, exact). Front-end entries show the *locked* version from `package-lock.json`; where the declared range differs it is given in Table 18.2.

Table 18.1: Master technology stack

Category	Technology	Version	Purpose
Frontend	Next.js	14.2.35	UI and backend-for-frontend
Frontend	React	18.3.1	Component rendering
Frontend	TypeScript	5.9.3	Typed front-end source
Backend	ASP.NET Core	10.0.x	HTTP API, hubs, health
Language	C#	14.0	Server, agent, tooling
Runtime	.NET	net10.0	Application runtime
Runtime	Node.js	20	Front-end server
Framework	MediatR	12.4.1	CQRS dispatch
Framework	Entity Framework Core	10.0.10	ORM and migrations
Framework	FluentValidation	11.11.0	Request validation
Framework	Hangfire	1.8.24	Background jobs
Library	Mapster	7.4.0	Object mapping
Library	QuestPDF	2024.10.3	PDF documents
Library	ZXing.Net	0.16.9	Barcodes
Library	ClosedXML	0.104.2	Excel export
Library	CsvHelper	33.0.1	CSV import/export
Library	YamlDotNet	16.2.1	YAML parsing
Library	System.IO.Ports	10.0.10	Serial hardware access
UI Library	Radix UI	1.x–2.x	Accessible primitives
UI Library	lucide-react	0.468.0	Icons
UI Library	recharts	2.15.4	Charts
UI Library	Tailwind CSS	3.4.19	Utility CSS
State	TanStack Query	5.101.4	Server-state cache
State	Zustand	5.0.14	Client state
Database	Microsoft SQL Server	2022 (image)	Primary data store
Database	SQLite	via Microsoft. Data. Sqlite 10.0.10	Agent offline spool
Cache	Redis	7 (image)	Optional cache and SignalR backplane
Authentication	ASP.NET Core Identity	10.0.10	Users, roles, logout
Authentication	OpenIddict	7.6.0	OAuth2/OIDC for machine clients

continued on next page

Category	Technology	Version	Purpose
Authentication	JwtBearer	10.0.10	Staff and shopper schemes
Authentication	Argon2 (Konscious)	1.3.1	PIN hashing
API	Swashbuckle.AspNetCore	10.2.3	OpenAPI/Swagger
API	SignalR	10.0.10	Realtime hubs
API client	axios	1.18.1	Browser HTTP
API client	@microsoft/signalr	8.0.17	Browser realtime
Build Tool	dotnet CLI / MSBuild	.NET 10 SDK	Build and publish
Build Tool	Next.js build	14.2.35	Standalone front-end bundle
Package Manager	NuGet (central)	N/A	.NET dependencies
Package Manager	npm	lockfile v3	Front-end dependencies
Testing	xUnit	2.9.3	All .NET suites
Testing	FluentAssertions	6.12.2	Assertions
Testing	NSubstitute	5.3.0	Test doubles
Testing	CsCheck	4.2.0	Property-based tests
Testing	NetArchTest.Rules	1.3.2	Dependency rules
Testing	Testcontainers (MsSql, Redis)	4.0.0	Real dependencies in tests
Testing	Playwright	1.62.0	End-to-end browser tests
Testing	Vitest	2.1.9	Front-end unit tests
Observability	Serilog	10.0.0	Structured logging
Observability	OpenTelemetry	1.17.0	Traces and metrics
Deployment	Docker	Not Verified	Images for API and web
Deployment	GitHub Actions	N/A	CI and release
Hosting	myASP.NET (shared Windows)	N/A	Production host for <code>pos.sma-techno.net</code>
Hosting	Caddy / IIS	Not Verified	Reverse proxy and IIS configuration provided
Benchmarking	BenchmarkDotNet	0.14.0	Performance harness

18.1 Front-end dependency inventory

Table 18.2: Front-end dependencies: declared range versus locked version

Package	Declared	Locked	Type
next	14.2.35	14.2.35	runtime
react	^18.3.1	18.3.1	runtime
react-dom	^18.3.1	18.3.1	runtime
@microsoft/signalr	^8.0.7	8.0.17	runtime
@tanstack/react-query	^5.62.0	5.101.4	runtime
@tanstack/react-query-devtools	^5.62.0	5.101.4	runtime
@tanstack/react-virtual	^3.14.9	3.14.9	runtime
@radix-ui/react-dialog	^1.1.4	1.1.23	runtime
@radix-ui/react-dropdown-menu	^2.1.4	2.1.24	runtime
@radix-ui/react-label	^2.1.1	2.1.15	runtime
@radix-ui/react-select	^2.1.4	2.3.7	runtime
@radix-ui/react-separator	^1.1.1	1.1.15	runtime
@radix-ui/react-slot	^1.1.1	1.3.3	runtime

continued on next page

Package	Declared	Locked	Type
@radix-ui/react-tabs	^1.1.2	1.1.21	runtime
@radix-ui/react-toast	^1.2.4	1.2.23	runtime
@radix-ui/react-tooltip	^1.1.6	1.2.16	runtime
axios	^1.7.9	1.18.1	runtime
zustand	^5.0.0	5.0.14	runtime
recharts	^2.15.0	2.15.4	runtime
lucide-react	^0.468.0	0.468.0	runtime
date-fns	^4.1.0	4.4.0	runtime
jose	^5.10.0	5.10.0	runtime
clsx	^2.1.1	2.1.1	runtime
tailwind-merge	^2.6.0	2.6.1	runtime
class-variance-authority	^0.7.1	0.7.1	runtime
typescript	^5.7.3	5.9.3	dev
tailwindcss	^3.4.17	3.4.19	dev
postcss	^8.4.49	8.5.23	dev
autoprefixer	^10.4.20	10.5.4	dev
eslint	^8.57.0	8.57.1	dev
eslint-config-next	14.2.21	14.2.21	dev
vitest	^2.1.9	2.1.9	dev
@playwright/test	^1.62.0	1.62.0	dev
@types/node	^22.10.5	22.20.1	dev
@types/react	^18.3.18	18.3.31	dev
@types/react-dom	^18.3.5	18.3.7	dev

Evidence file for every row: `frontend/package.json` (declared) and `frontend/package-lock.json` (locked).

Chapter 19

Technology Interaction Matrix

Table 19.1: Component interactions

Source	Target	Method	Purpose	Evidence
Browser	Next.js server	HTTPS	Pages and BFF routes	src/app/
Browser	SignalR hubs	WebSocket + ticket	Live tag and grid updates	src/lib/rfid-hub.ts
BFF route handlers	API	HTTP, server-to-server	Proxy with token attached	app/api/proxy/[...path]/route.ts:87
BFF sign-in	API /auth/token	HTTP POST	Exchange credentials for tokens	lib/server/auth-api.ts:49
BFF	API /auth/refresh	HTTP POST	Refresh an expired token	lib/server/auth-api.ts:111
API controllers	Application	MediatR	Dispatch commands and queries	Retail25.Api/Controllers/
Application	SQL Server	EF Core / SQL	Persistence	DependencyInjection.cs:83
Application	Cache provider	Port interface	Carts, debounce, idempotency	Cache:Provider
Application	SignalR hubs	IPosNotifier, IRfidNotifier	Push state changes	Infrastructure/Realtime/
Hangfire	Application	Job invocation	Accrual and accounting posting	Program.cs:439--445
Terminal agent	/hubs/terminal	SignalR (outbound)	Publish tag batches	TerminalAgent/Server/ServerConnection.cs
Terminal agent	API /api/v1/terminals/...	HTTP	Profile and device configuration	Server/ProfileRefreshService.cs
Terminal agent	RFID reader	TCP or serial	Inventory tags	Retail25.Devices/Rfid/
Terminal agent	SQLite spool	File I/O	Survive a server outage	Spooling/SqliteTagSpool.cs
API	OTLP collector	gRPC/HTTP	Traces and metrics	deploy/otel/config.yaml

continued on next page

Source	Target	Method	Purpose	Evidence
CI	GHCR	Docker push	Publish images from <code>main</code>	<code>ci.yml</code>

Chapter 20

Repository Evidence and Verification Notes

20.1 Where this document was read from

Branch main, worktree .claude/worktrees/phase-3-4-completion-0addae, commit e7f91ad.

This matters. The repository's *default* checkout at C:\Users\GT\Desktop\Retail25 is on branch feat/pricing-engine-and-runnable-stack at commit 24700f4 with only three commits of history. That checkout is substantially behind. All findings here come from the worktree on main.

20.2 Principal files inspected

File	What it established
backend/Directory.Build.props	Target framework, language version, warning policy, product version
backend/Directory.Packages.props	Every .NET package version; security pins
backend/Retail25.sln	Project inventory
backend/src/Retail25.Api/Program.cs	Host configuration, hubs, health, rate limits, jobs
backend/src/Retail25.Infrastructure/DependencyInjection.cs	SQL Server provider, naming convention, Hangfire storage
.../Identity/IdentityRegistration.cs	OpenIddict flows, JWT schemes, certificate policy
.../Controllers/StaffAuthController.cs	Staff token endpoint and logout behaviour
.../Realtime/RfidHub.cs	Hub groups and notification targets
.../Persistence/ApplicationDbContext.cs	89 entity sets
frontend/package.json, package-lock.json	Declared and locked front-end versions
frontend/next.config.js	Strict mode, standalone output
frontend/src/lib/server/session.ts	Cookie names, flags, sealing
frontend/src/app/api/...	BFF route inventory
.github/workflows/ci.yml	Four CI jobs
.github/workflows/deploy-myasp.yml	Five release jobs
deploy/docker-compose.yml, Dockerfile.*	Images, ports, service topology
README.md	Project purpose, stack summary, ledger stance

20.3 Documentation divergences found

These are recorded because they materially mislead a reader of the existing documentation.

Topic	Documentation says	Code shows
Database	CLAUDE.md: DBF files "replaced by PostgreSQL"; docs/architecture/ README.md: "PostgreSQL 16"	SQL Server. <code>UseSqlServer</code> ; only the SqlServer EF provider is referenced. README.md acknowledges the swap.
Staff authentication	README.md and docs/ architecture/07: "Authorization Code + PKCE, BFF cookies"	A custom JWT endpoint, <code>POST /auth/ token</code> , using Identity's <code>CheckPasswordSignInAsync</code> . OpenIddict's code flow is still enabled but is not what staff use.
Parity benchmark	CLAUDE.md: docs/ BENCHMARK.md is authoritative, "71 of 155 delivered"; capabilities. json drives it	Neither file exists on this branch. <code>Retail25.ArchitectureTests</code> holds only dependency-rule and transport-security suites.
Runtime version	docs/architecture/ README.md: ".NET 8 LTS", "EF Core 8", "OpenIddict 5.x"	<code>net10.0</code> , EF Core 10.0.10, OpenIddict 7.6.0.
Redis	docs/architecture/ README.md: "Cache/Debounce: Redis"	Optional. <code>Cache:Provider</code> defaults elsewhere; the Compose file provides Redis but the code treats it as opt-in.
CLAUDE.md location	Referenced as project guidance	Not present on <code>main</code> ; it exists on the stale <code>feat/...</code> branch.

20.4 Not found, not configured, or unable to verify

- **Form and schema-validation libraries** on the front end — none declared.
- **Prettier or a .NET formatter configuration** — not found.
- **Accessibility testing tooling** — not configured.
- **Parity benchmark artefacts** — absent on this branch.
- **Capacity figures** (CPU, RAM, storage, bandwidth) — no manifest, autoscaling policy or load-test result states any; deliberately not estimated.
- **Current test pass rates** — not executed as part of this document.
- **Automated backup schedule** — a restore runbook exists; a schedule does not.

20.5 Inference, flagged

Statements in this document marked **Derived** are inferences from implementation, chiefly: that readers must be reachable from the till's network (because the agent opens the connection), that tills need only outbound access (because the agent is a SignalR client), and that persistent volumes are required (because Compose declares them). None is stated as a project requirement.

Chapter 21

Summary and Next Steps

21.1 The confirmed stack

C# on .NET 10 with Clean Architecture and CQRS through MediatR; SQL Server via EF Core 10 with explicit migrations and snake-case naming; ASP.NET Core for the HTTP API, four SignalR hubs, health checks, per-caller rate limiting and Hangfire jobs; ASP.NET Core Identity with a custom staff JWT endpoint, separate shopper and machine-client schemes, and OpenIddict for the latter; Next.js 14 with React 18, TypeScript, Tailwind and Radix, acting as both UI and backend-for-frontend; and a Windows-service terminal agent bridging RFID readers and till peripherals over an outbound SignalR connection.

21.2 Architectural characteristics worth noting

- **Ledgers, not mutable rows.** Derived snapshots over append-only history.
- **No token in the browser.** The session is a sealed, `httpOnly` cookie; the proxy attaches tokens server-side; hubs use single-use tickets.
- **Configuration over code.** Permissions, tax, tender types, pricing rules and reader profiles are rows, not literals.
- **Fail loudly when unconfigured.** The API refuses to start on a placeholder origin or without signing keys, rather than running in a quietly wrong state.
- **Dependency direction is tested,** not merely documented.

21.3 What is missing or unverified

Capacity requirements, current test results, an automated backup schedule, and — most significantly — the parity-tracking mechanism that the project's own guidance treats as the definition of "delivered".

21.4 Recommended next steps

These are recommendations, not existing capabilities.

1. **Reconcile the documentation with the code,** starting with the three divergences in Chapter 20. An engineer following `docs/architecture/` today would stand up the wrong database and expect the wrong auth flow.
2. **Restore or retire the parity benchmark.** Either bring `docs/BENCHMARK.md` and `capabilities.json` onto main, or remove the claim that they govern delivery.

-
3. **Establish capacity by measurement.** A short load test against the integration environment would replace the only genuinely unknown section of this document.
 4. **Decide the Redis question explicitly.** Under the SQL Server cache provider there is no SignalR backplane and therefore a single-instance ceiling. That is a scaling limit worth stating in the deployment documentation.
 5. **Make `npm audit blocking`,** or record why it is advisory.